# Separação de Responsabilidades: Modal, Componentes de Apresentação e Formulário Validado com React

**Repositório:** https://github.com/Michel-Azvdo/faith-lettersFD

## 1. Visão geral

Nesta atividade, implementei no projeto Faith Letters um modal reutilizável para o envio de cartas a missionários. Separei a interface (componentes de apresentação) da lógica (componentes de controle) e usei React Hook Form com Zod para validar o formulário.

## 2. Arquitetura: Smart e Dumb Components

**Modal (Dumb Component).** É responsável apenas pela interface. Recebe `isOpen`, `onClose`, `title` e `children` por props e desenha a caixa de diálogo. Não busca dados, não conhece cartas nem missionários e não guarda estado de negócio. Só avisa, por `onClose`, quando o usuário quiser fechar (botão X, clique no fundo ou tecla Esc). Por isso pode ser reaproveitado em qualquer tela do projeto.

**CreateLetterModal (Smart Component).** É quem sabe o que acontece. Usa o Modal para exibir o conteúdo e concentra a lógica: lê os missionários do `AppContext`, busca o missionário escolhido pelo ID, monta o objeto da carta, salva pela API (`saveLetterToAPI`), atualiza a lista de cartas e controla os estados de sucesso (`showSuccess`) e de erro.

**LetterForm.** Mostra os campos (missionário, nome e mensagem), exibe os erros de validação abaixo de cada campo e limpa o formulário depois de um envio bem-sucedido. Recebe tudo por props (`missionaries`, `onSubmit`, `onCancel`) e não sabe como a carta é salva.

**App.tsx.** Mantém o estado `isModalOpen` e o passa ao CreateLetterModal. O botão “Enviar carta” do cabeçalho altera esse estado.

**Por que essa separação?** Cada arquivo tem uma única responsabilidade. Isso facilita a manutenção (mudar o visual não afeta a lógica e vice-versa), permite reaproveitar o Modal em outros contextos e deixa o código mais fácil de testar e de entender.

## 3. Validação com Zod e React Hook Form

**Zod (`letterSchema.ts`).** Define em um só lugar as regras do formulário: `missionaryId`, `senderName` e `content` são obrigatórios, cada um com sua mensagem de erro. O tipo

`LetterForm Data` é gerado automaticamente do schema com `z.infer`, o que evita escrever o tipo duas vezes e mantém validação e TypeScript sincronizados.

**React Hook Form.** Gerencia o estado do formulário com `register`, conectando cada campo sem precisar de um `userState` por input. O `zod Resolver` liga o schema ao formulário, e a função de envio só é chamada quando todos os dados são válidos. Os erros ficam disponíveis em `formState.errors` e são exibidos abaixo de cada campo. Também usei `noValidate` no formulário para que a validação seja feita pelo Zod, e não pelo navegador.

## 4. Processamento e feedback

Ao enviar, o `handleSubmit` do CreateLetterModal:

1. Busca o missionário selecionado pelo ID.
2. Monta a carta com nome do remetente, conteúdo, ID do missionário e data.
3. Salva a carta pela API (json-server).
4. Atualiza a lista de cartas da tela.
5. Exibe uma mensagem de sucesso, que desaparece sozinha após 4 segundos com `setTimeout` dentro de um `useEffect`.

O LetterForm só limpa os campos se o envio der certo. Se ocorrer um erro, o texto digitado é mantido e uma mensagem de erro aparece, para o usuário não perder o que escreveu.

## 5. Estilização e acessibilidade

Usei CSS Modules para que os estilos de cada componente fiquem isolados. Pensando em responsividade e acessibilidade, implementei:

- layout adaptado para celular, com o modal ocupando a tela e os botões empilhados;
- fechamento do modal com a tecla Esc e bloqueio da rolagem da página ao fundo;
- atributos `role="dialog"`, `aria-modal`, `aria-labelledby` e `aria-label` no botão de fechar;
- `aria-invalid` e `role="alert"` nos campos com erro;
- contorno de foco visível para navegação por teclado;
- respeito à preferência de redução de movimento (`prefers-reduced-motion`).

## 6. Testes realizados

- Envio com o formulário vazio: os três erros aparecem.
- Envio com dados válidos: a carta é salva, os campos são limpos, a mensagem de sucesso aparece e depois some.
- A nova carta aparece na lista de cartas e é gravada no `db.json`.
- Fechamento do modal pelo X, pelo clique no fundo e pela tecla Esc.
- Navegação apenas pelo teclado e visualização em tamanho de celular.

## 7. Conclusão

A atividade me ajudou a entender na prática por que separar lógica e apresentação deixa o código mais organizado e reutilizável, e como React Hook Form com Zod simplifica a validação de formulários e o controle de estado. O código está versionado no GitHub e será expandido nas próximas unidades.